学术AI鉴伪系统概要设计说明书（迭代版）

(AID-SD-1)

概要设计说明书

<table>
<tr><td rowspan="4">文件状态：
[✓] 草稿
[ ] 正式发布
[ ] 正在修改</td><td>文件标识：</td><td>AID-SD-1</td></tr>
<tr><td>当前版本：</td><td>v0.1</td></tr>
<tr><td>作　　者：</td><td>刘宇童　聂薪羽　苏天
翟润　张与耕</td></tr>
<tr><td>完成日期：</td><td>2026 年 4 月 9 日</td></tr>
</table>

目录

1 引言 **3**
1.1 编写目的 3
1.2 文档范围 3
1.3 参考资料 3
1.4 术语说明 3

2 系统概述 **3**
2.1 设计背景 3
2.2 现有系统基础 4
2.3 现有系统与迭代目标差异 4
2.4 设计目标 5
2.5 用户与角色视角 5
2.6 业务范围 6
2.7 设计原则 6
2.8 系统边界 6

3 总体架构设计 **7**
3.1 总体架构说明 7
3.2 架构选型依据 7
3.3 系统总体静态结构类图 8
3.4 统一资源与检测扩展关系类图 9
3.5 系统组成 9
3.6 子系统职责划分 10
3.7 核心调用链设计 10
3.8 分层设计 11
3.9 架构约束与关键设计决策 11
3.10 面向迭代的架构调整方向 12
3.11 关键架构优势 12
3.12 总体架构小结 12

4 模块设计 **12**
4.1 模块划分原则 12
4.2 模块总体关系类图 13
4.3 模块总览 13
4.4 用户与权限模块 14
4.5 组织与成员模块 14
4.6 资源管理模块 15
4.7 检测任务管理模块 15
4.8 图像检测模块 15
4.9 论文检测模块 16
4.10 评审检测模块 16
4.11 人工审核模块 16

4.12 报告服务模块 17
4.13 通知消息模块 17
4.14 统计日志模块 17
4.15 模块协作关系说明 17
4.16 本章小结 18

5 数据设计 18
5.1 数据设计目标 18
5.2 核心实体 18
5.3 实体关系说明 18
5.4 状态设计 18
5.5 兼容与演进说明 19

6 接口设计 19
6.1 接口设计原则 19
6.2 外部接口分类 19
6.3 内部接口分类 19
6.4 接口分层说明 19

7 关键流程设计 19
7.1 资源上传与检测流程 19
7.2 图像检测流程 20
7.3 论文检测流程 20
7.4 Review检测流程 20
7.5 人工审核流程 20
7.6 通知与状态同步流程 20
7.7 流程图说明 20

8 运行与部署设计 20
8.1 部署结构 20
8.2 运行机制 20
8.3 部署约束 21

9 质量属性设计 21
9.1 性能设计 21
9.2 安全设计 21
9.3 可维护性设计 21
9.4 可扩展性设计 21
9.5 可靠性设计 21

10 设计约束与待定问题 22
10.1 当前约束 22
10.2 待定问题 22
10.3 后续工作 22

1 引言

1.1 编写目的

本文档用于描述学术AI鉴伪系统在迭代开发阶段的概要设计方案，明确系统的总体结构、模块划分、关键数据关系、主要交互流程以及部署与质量设计，为后续详细设计、编码实现、测试验证和系统演进提供依据。

1.2 文档范围

本文档重点覆盖以下内容：

- 迭代后系统的总体架构与模块边界；
- 用户端、管理端、后端服务与AI推理服务之间的协作关系；
- 学术图像检测、全篇论文检测、Review检测、人工审核与报告生成的设计方案；
- 核心数据实体、接口分层、关键业务流程与运行部署设计；
- 可维护性、可扩展性、安全性与性能等质量属性设计。

1.3 参考资料

- 《学术AI鉴伪系统需求规格说明书（迭代版）》；
- 当前项目代码仓库中的前端、后端与AI服务实现；
- 项目现有接口文档、实体表与交接说明。

1.4 术语说明

术语	说明
学术资源	系统支持处理的业务对象，包括学术图像、论文文件与同行评审Review。
检测任务	用户针对一个或多个学术资源发起的一次检测处理请求。
检测结果	系统对指定资源执行自动检测后产生的结构化输出。
人工审核	在自动检测基础上，由人工参与复核、判断或补充说明的业务过程。
综合报告	汇总自动检测与人工审核信息后形成的可下载输出结果。

2 系统概述

2.1 设计背景

学术内容真实性识别已不再局限于传统图像篡改检测场景。随着生成式人工智能技术在论文写作、图片生成与学术评审文本中的广泛使用，学术不端风险逐渐呈现出多类型、跨环节与高隐蔽性的特点。对于期刊、科研机构和研究团队而言，仅依靠单一内容类型、单一检测模型或人工抽查方式，已难以满足对学术资源进行系统化风险识别的需求。

当前项目已完成一轮以学术图像检测为核心的系统建设，具备用户注册登录、图像上传、图片提取、图像自动检测、人工审核、结果通知以及管理端统计等基础能力。从代码实现上看，系统已经形成了由用户端、管理端、Django后端、异步任务系统和独立AI推理服务组成的可运行基础框架，为后续迭代提供了较好的工程基础。

然而，现有系统的业务抽象和数据模型仍明显围绕图像检测链路展开，尚不能完整支撑迭代需求中提出的全篇论文检测、Review检测、综合报告统一生成、模型配置治理以及论文/Review专项日志管理等能力。因此，本轮概要设计的核心任务不是简单补充页面或接口，而是在保留现有可用能力的基础上，完成系统从“图像检测平台”向“综合学术AI鉴伪平台”的结构升级。

2.2 现有系统基础

结合当前项目代码，现有系统已经具备如下实现基础：

- 在前端层面，已分别实现用户端与管理端两套Web应用，能够支持基本业务操作、历史记录查询、人工审核入口与管理统计展示；
- 在后端层面，已形成以Django为核心的业务服务，提供用户认证、资源上传、检测任务管理、人工审核、组织管理、通知与管理端接口；
- 在异步处理层面，已集成Celery与Redis，支持检测任务异步执行、批处理调度、任务状态更新与结果回写；
- 在AI推理层面，已具备独立的图像推理服务和图像检测流水线，能够执行图像伪造识别、可疑区域生成与结果封装；
- 在数据层面，已具备用户、组织、上传文件、检测任务、检测结果、人工审核、日志与通知等核心表结构。

上述基础说明，当前系统并非从零开始设计，而是在已有实现上进行面向迭代需求的重构与扩展。因此，概要设计需要同时考虑两个方面：一方面要充分复用已有实现中成熟的模块和机制；另一方面要识别现有设计中图像中心化、模型耦合和实体抽象不足等问题，并提出统一化、可扩展的改造方案。

2.3 现有系统与迭代目标差异

从需求文档与现有实现对照来看，当前系统与本轮目标系统之间主要存在以下差异：

对比项	当前系统现状	本轮迭代目标
核心检测对象	以学术图像为主	统一支持学术图像、全篇论文与Review
业务抽象中心	以图像上传和图像结果为中心	以统一学术资源、统一检测任务和统一报告为中心
AI能力组织方式	图像检测链路已落地，模型调用与业务耦合较强	不同检测对象对应不同检测适配器，模型调用更加配置化与模块化
结果输出方式	以图像检测结果和图像报告为主	统一支持图像、论文、Review结果展示与综合报告生成

人工审核范围	以图像审核流程为主	需要与更广义的综合鉴伪流程进行衔接
日志与治理能力	已具备操作日志和部分统计能力	进一步增加论文日志、Review日志、模型配置和资源治理能力

因此，本轮概要设计的重点不在于重写已有图像检测流程本身，而在于建立能够承载三类学术资源的总体结构，使现有功能能够作为其中一个可复用子链路纳入统一架构。

2.4 设计目标

本轮概要设计的总体目标如下：

- 在保留现有学术图像检测能力的前提下，将系统扩展为面向学术图像、全篇论文和Review的统一鉴伪平台；
- 对上传、检测、审核、报告、日志与后台治理形成一致的系统结构，避免不同资源类型各自独立演化；
- 降低现有实现中图像中心化带来的耦合问题，为新增资源类型与新增检测模型预留稳定扩展点；
- 明确前端、后端、异步任务系统、AI推理服务、数据库和文件存储之间的职责边界与协作关系；
- 为后续详细设计、接口扩展、数据库调整和模块重构提供统一依据。

进一步地，从工程交付角度看，本轮设计还应满足以下目标：

- 能够支撑逐步迭代开发，而不是要求一次性完成全部重构；
- 能够兼容已有图像检测代码与数据结构，降低重构风险；
- 能够将新增论文检测和Review检测纳入现有任务调度、通知和管理体系中；
- 能够支撑课程项目背景下的文档撰写、演示展示和后续验收。

2.5 用户与角色视角

根据需求文档和当前实现，本系统主要服务于以下三类角色：

- **普通用户**：主要包括科研人员、学生、编辑和其他资源提交者。其关注点在于上传资源、发起检测、查看结果、下载报告以及申请人工审核；
- **审核参与者**：主要指具有人工审核职责的用户。其关注点在于接收审核任务、查看资源与自动检测结果、提交审核结论及原因说明；
- **管理员**：包括组织管理员和超级管理员。其关注点在于组织和用户治理、审核审批、资源管理、日志查询、统计分析和模型配置。

不同角色共享同一业务平台，但其功能入口、数据访问范围和操作边界不同。因此，概要设计中需要明确角色分层，而不能将系统视为单一用户形态的应用。

2.6 业务范围

结合需求规格说明书，本轮迭代后系统的业务范围主要包括以下几个部分：

- **资源接入范围**：系统支持图像文件、论文文件和Review文本三类资源进入统一检测流程；
- **自动检测范围**：系统支持对图像伪造风险、论文AIGC风险和Review模板化/AI生成倾向进行自动分析；
- **结果输出范围**：系统支持检测结果展示、结构化结果查询与综合报告生成；
- **人工复核范围**：系统支持用户发起人工审核请求，并在必要时引入审核参与者和管理员共同完成复核流程；
- **后台治理范围**：系统支持对用户、组织、资源、日志、审核和模型进行统一管理。

2.7 设计原则

为了保证本轮系统设计既能兼容当前实现、又能支撑后续演进，概要设计遵循以下原则：

- **模块化原则**：按业务职责划分模块，减少不同模块之间的直接耦合；
- **统一抽象原则**：对不同类型学术资源采用统一任务与结果抽象，不再仅围绕图像对象组织系统；
- **异步处理原则**：长时检测任务通过异步机制执行，避免阻塞用户交互主流程；
- **可扩展原则**：为新增检测对象、检测模型和管理能力预留扩展接口；
- **可追溯原则**：关键业务过程保留任务状态、日志与报告映射关系，保证结果可回查；
- **兼容演进原则**：在设计上兼顾现有系统可复用部分与目标结构升级，避免一次性推翻现有实现。

2.8 系统边界

从系统边界看，本系统主要由用户端、管理端、后端业务服务、异步任务处理服务、AI推理服务、数据库和文件存储构成。系统对外提供Web访问与业务接口能力，对内通过HTTP调用、消息队列、数据库记录和文件共享完成各组件之间的协作。

系统边界之外的内容主要包括：

- 具体检测模型的训练过程与训练平台；
- 第三方学术数据库或外部内容平台的直接接入；
- 大规模生产级运维体系中的弹性调度与云原生编排能力；
- 不属于本课程项目实现范围的深度审计或合规系统。

也就是说，本文档关注的是系统如何组织、调用和管理检测能力，而不是对底层模型训练机制或外部科研生态系统做全面设计。

3 总体架构设计

3.1 总体架构说明

系统总体上采用“前后端分离 + 统一业务服务 + 异步任务调度 + 独立AI推理服务”的组合架构。该架构不是脱离现有工程基础的重新假设，而是在当前项目代码已经形成用户端、管理端、Django 后端、Celery 异步任务、Redis 消息中转和独立图像推理服务的基础上，对未来迭代目标进行整理和提升后的结果。

从业务演进方向看，系统已经不再只是处理图像伪造检测，而是需要统一承载学术图像、全篇论文和 Review 文本三类检测对象。因此，体系结构必须从“围绕图像功能组织实现”过渡为“围绕统一资源、统一任务和统一结果组织实现”。前端负责交互与展示，后端负责业务编排，异步任务系统负责长耗时检测任务的排队和执行，AI 推理服务负责具体模型运行，数据库与文件存储负责持久化和追溯。通过这种职责拆分，可以在保留现有图像链路的同时，为论文检测、Review 检测和综合报告生成预留稳定扩展点。

需要说明的是，本章为了强调系统静态结构和依赖关系，统一采用 UML 类图风格描述主要子系统和核心抽象，而不使用传统流程框图。这里的“类”并不局限于面向对象代码类，而是作为对系统中稳定职责单元的结构化表达。

3.2 架构选型依据

本系统选择当前总体架构，主要基于以下考虑：

- **Web 访问是主要交互方式**。系统主要面向浏览器端用户，用户端和管理端存在明显不同的页面需求与权限边界，采用前后端分离结构更有利于界面独立演进；
- **检测任务执行时间较长**。无论是图像检测、论文检测还是 Review 检测，都可能涉及文件解析、特征提取、批量切分和模型推理，不适合同步阻塞式请求处理；
- **检测对象存在明显差异**。图像、论文和 Review 的输入格式、预处理方式、结果结构和模型调用方式并不一致，需要通过统一抽象之上的适配机制来隔离差异；
- **系统需要持续迭代扩展**。本轮只是综合学术 AI 鉴伪平台的第一阶段扩展，后续仍可能增加模型管理、报告策略、组织治理和更细粒度的人工复核能力，因此架构必须支持增量演进；
- **当前代码已具备可复用基础**。继续整理和提升现有 Django、Celery、Redis、WebSocket 与独立 AI 服务体系，相比完全推翻重写更具工程可行性。

3.3 系统总体静态结构类图

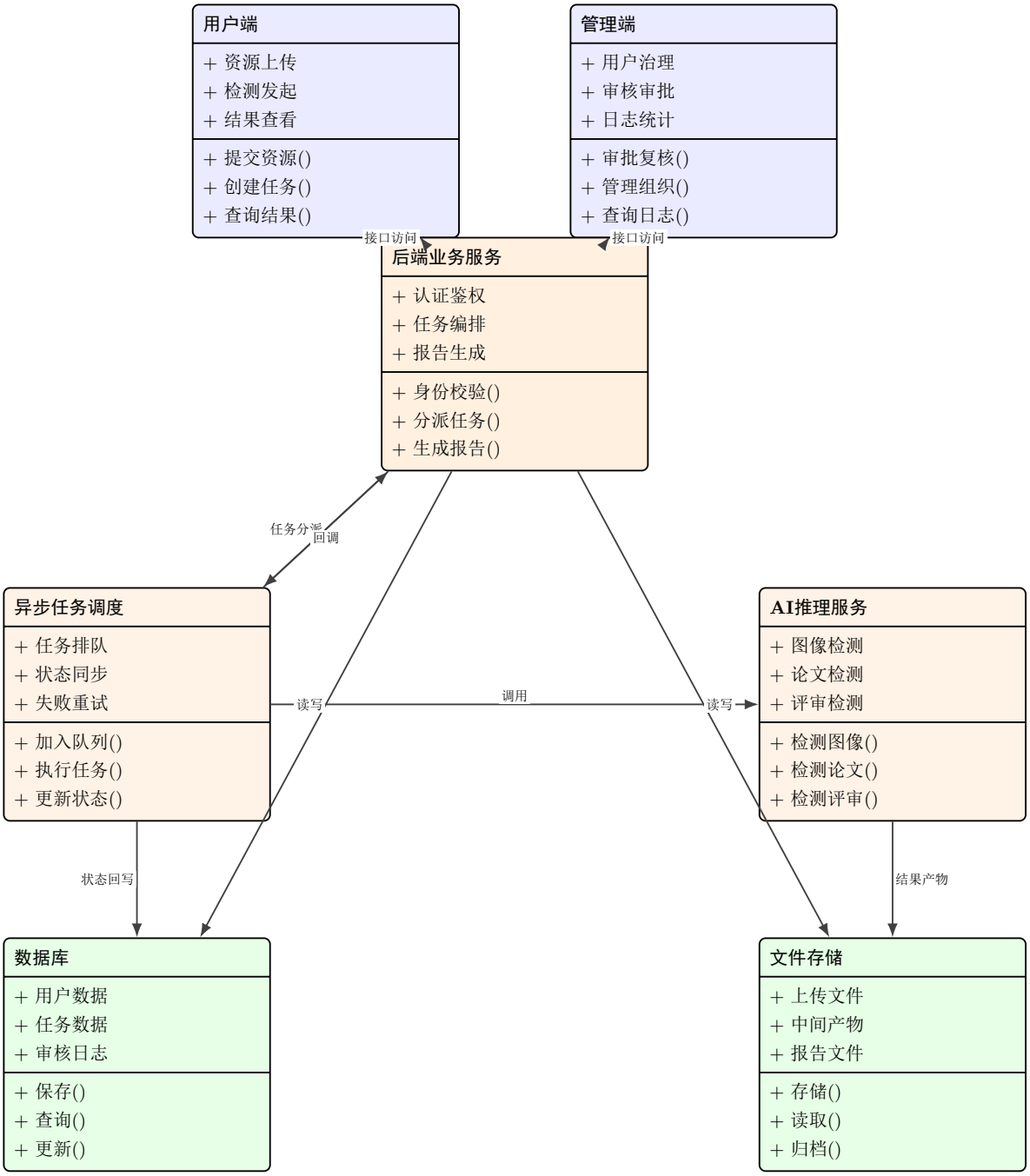

图 1: 系统总体静态结构 UML 类图

图 3-1 从系统静态依赖关系角度展示了本项目的总体结构。用户端和管理端分别面向普通用户与管理用户，通过统一后端业务服务访问平台能力；后端业务服务一方面直接管理认证鉴权、组织与资源治理、结果查询和报告生成等同步能力，另一方面通过异步任务调度子系统将长耗时检测请求提交到 AI 推理服务；推理结果再由业务服务统一汇总并落入数据库与文件存储，最终由通知与状态反馈机制返回到前端界面。

与传统的分层框图相比，使用类图样式表达的好处在于可以更清楚地表示各个稳定子系统之间的静态关联关系、依赖方向和职责边界，便于后续详细设计时继续向模块类图和实体类图细化。

3.4 统一资源与检测扩展关系类图

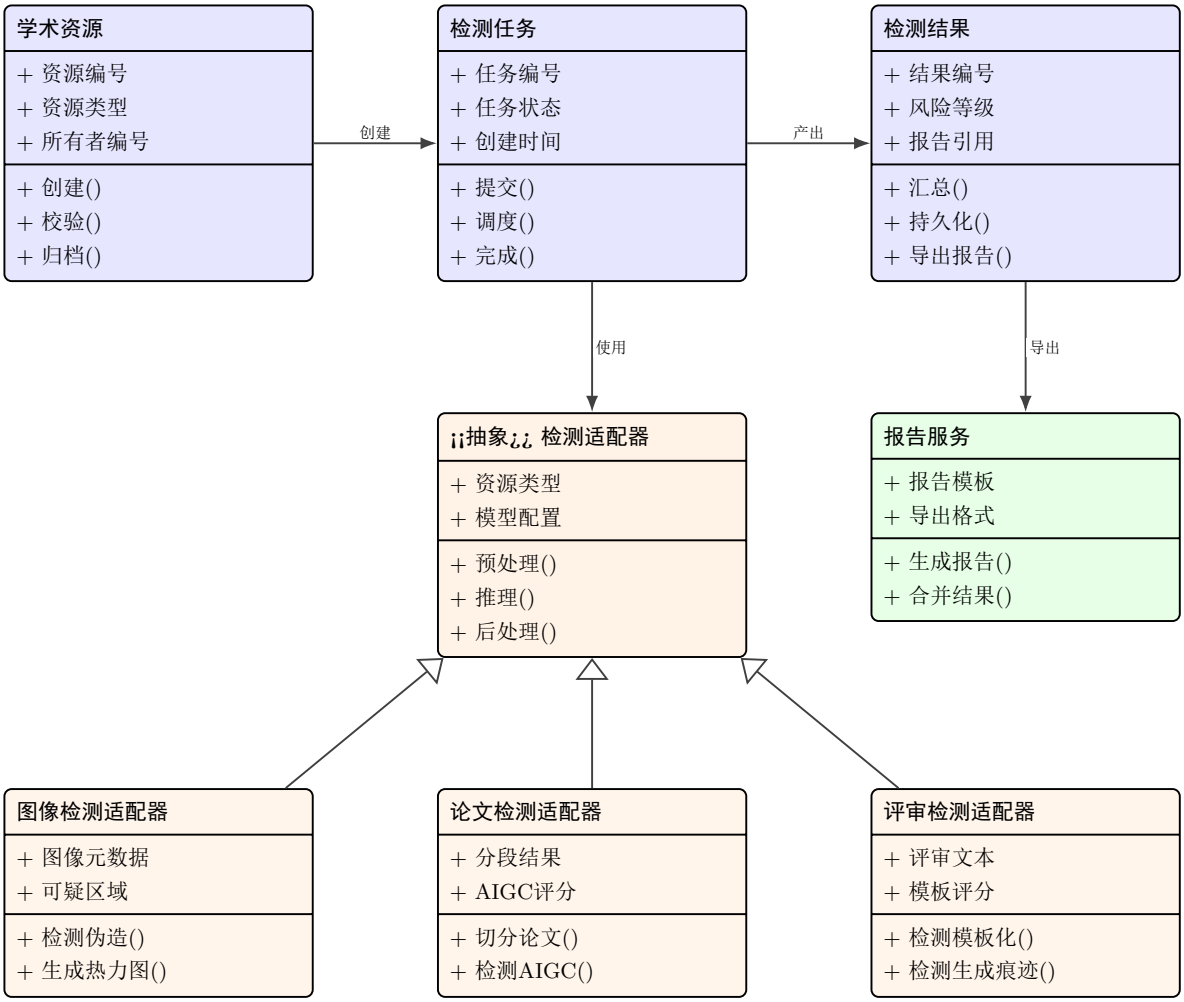

图 2: 统一资源与检测扩展关系 UML 类图

图 3-2 展示了本轮迭代在架构抽象层面最关键的一次调整，即将现有系统中偏图像中心的实现方式，提升为统一资源与统一检测任务的结构。具体而言，系统通过 **AcademicResource** 抽象统一承载图像、论文和 Review 三类对象，通过 **DetectionTask** 抽象统一管理检测请求和状态流转，通过 **DetectionResult** 抽象统一组织检测输出，并通过 **DetectionAdapter** 抽象隔离不同资源类型对应的模型调用差异。

这一设计的直接收益在于：后续新增一种检测对象时，不必改动整个任务链路和结果链路，只需要在统一任务框架下补充新的资源表示和适配器实现即可。这种方式比在现有图像任务逻辑上叠加条件分支更稳定，也更适合课程项目后续继续迭代。

3.5 系统组成

从逻辑结构上看，系统由以下几个核心组成部分构成：

- **用户端**：面向普通用户和审核参与者，负责资源上传、检测任务发起、结果查看、人工审核参与和通知展示；
- **管理端**：面向组织管理员和超级管理员，负责用户治理、组织管理、审核审批、日志查看、统

计分析和模型配置入口；

- **后端业务服务**：负责认证鉴权、资源管理、检测任务创建、结果聚合、报告生成、权限控制和管理接口暴露；
- **异步任务调度子系统**：负责长耗时任务的排队、拆分、执行、失败重试和状态同步；
- **AI 推理子系统**：负责图像、论文和 Review 等不同检测模型的具体执行；
- **消息与通知子系统**：负责检测状态推送、系统通知分发和用户消息反馈；
- **数据库与文件存储**：负责业务数据、日志、上传资源、检测中间产物和报告文件的持久化。

3.6 子系统职责划分

子系统	主要职责	典型技术实现
用户端子系统	面向普通用户和审核用户提供资源上传、检测发起、结果展示、审核参与和通知接收能力	Vue + Vuetify
管理端子系统	面向组织管理员和超级管理员提供治理、统计、日志查询和审核审批能力	Vue + Vuetify
业务服务子系统	提供统一 REST 接口，承担认证鉴权、权限控制、业务编排、结果封装和报告管理	Django + DRF
消息与实时通知子系统	负责 WebSocket 推送、任务状态广播、系统公告与用户通知分发	Django Channels
异步调度子系统	负责长时任务调度、批处理拆分、失败重试、执行收尾和状态回写	Celery + Redis
AI 推理子系统	负责检测模型推理、特征计算、风险分析和差异化结果生成	独立 Python 推理服务
数据存储子系统	负责业务数据、任务状态、日志记录、审核记录和元数据存储	MySQL
文件存储子系统	负责上传资源、结果文件、可视化产物和报告文件保存	本地媒体目录/文件系统

3.7 核心调用链设计

系统的核心调用链可概括为以下几个阶段：

1. 用户通过用户端或管理端发起业务请求，例如上传资源、提交检测、申请人工复核或审批审核任务；
2. 前端通过 HTTP API 或 WebSocket 将请求发送到后端业务服务；
3. 后端完成身份认证、权限校验、资源登记、任务创建和状态初始化，并根据任务性质决定同步处理还是异步处理；
4. 对于检测类长时请求，后端将任务提交到异步任务调度子系统，由调度子系统负责排队、分发和失败重试；

5. 异步任务调度子系统根据任务类型调用对应的检测适配器，由适配器进一步调用 AI 推理服务完成模型执行；
6. 推理结果经过后端业务服务统一汇总后写入数据库与文件存储，并触发报告生成、通知创建和状态变更；
7. 前端通过轮询接口或实时推送机制获取任务最新状态、检测结果和报告链接。

上述调用链将“用户请求响应速度”和“检测处理耗时”有效解耦，使平台既能保证交互上的及时反馈，又能处理较重的文件读写、模型推理和报告生成任务。

3.8 分层设计

为了使系统在结构上更清晰、在实现上更易维护，本系统采用分层设计。各层职责如下表所示：

层次	职责
表示层	提供用户端与管理端页面、表单交互、结果展示和状态反馈，仅负责界面组织与前端状态管理，不直接承载核心业务规则。
接口层	提供 HTTP API 与 WebSocket 接口，对外暴露统一访问入口，负责请求接收、参数校验和基础异常响应。
业务层	实现用户、组织、资源、检测、审核、报告、通知和统计等核心业务逻辑，是系统的业务中枢。
任务层	处理异步检测任务、批处理调度、状态同步、任务收尾和报告生成触发。
适配层	对不同资源类型对应的检测模型和外部推理能力进行适配与封装，屏蔽底层模型差异。
数据层	存储用户数据、任务数据、审核数据、日志、文件路径和报告元数据，保证业务数据可持久化和可追溯。

3.9 架构约束与关键设计决策

为了保证后续详细设计和编码实现能够沿同一方向演进，本系统在总体架构层面明确以下约束与设计决策：

- **前端不直接调用模型服务**。所有检测请求必须先进入后端业务服务，由后端统一完成权限校验、任务登记和状态管理；
- **检测必须通过统一任务中心流转**。无论检测对象是图像、论文还是 Review，原则上都应通过统一的任务创建、调度和回写机制执行；
- **结果主体与差异化载荷分离**。统一结果对象只保存通用状态、风险等级和报告引用，具体差异化内容由资源类型专属结构承载；
- **模型接入通过适配器隔离**。业务层不直接依赖具体模型实现，模型替换和模型扩展应限制在适配层及推理子系统内部；
- **人工审核链路独立于自动检测链路存在**。人工审核可以引用自动检测结果，但不应与模型执行过程耦合，从而保证人工复核流程可单独演进；

- **关键过程必须可追溯**。资源上传、任务创建、状态变化、审核结论和报告生成都需要在数据层保留可回查记录。

3.10 面向迭代的架构调整方向

考虑到当前实现仍以图像检测流程为中心，架构上需要逐步向统一资源架构过渡。该过渡主要体现在以下几个方面：

- **资源对象统一化**：从仅支持图像的资源表示过渡为统一支持图像、论文和 Review 的资源抽象；
- **检测能力适配化**：图像检测、论文检测和 Review 检测分别通过独立适配器接入，避免在业务代码中硬编码不同检测逻辑；
- **结果结构层次化**：采用统一结果主体与差异化结果载荷相结合的方案，使不同资源类型既能纳入统一任务流程，又能保留各自结果特点；
- **治理能力泛化**：管理端从围绕图像任务治理扩展为面向统一学术资源、统一日志和统一模型配置的治理结构；
- **报告生成标准化**：将现有偏图像结果的报告输出能力扩展为可面向多资源类型的统一报告服务。

3.11 关键架构优势

本轮总体架构设计具备以下优势：

- 能复用现有前后端、异步任务与图像检测链路，降低迭代开发成本；
- 能通过统一任务体系纳入论文和 Review 两类新资源，避免形成多个割裂子系统；
- 能将长时检测任务从用户请求链路中解耦，提高系统交互流畅性；
- 能将模型推理能力与业务编排逻辑分离，便于后续替换或新增模型；
- 能为后续详细设计中的数据库重构、接口扩展、实体建模和流程设计提供稳定基础。

3.12 总体架构小结

综上，本系统总体架构并不是对现有实现的简单罗列，而是对当前代码基础、迭代需求目标和后续演进方向进行综合平衡后的结果。其核心思想是：在保留当前图像检测平台工程基础的同时，通过统一资源、统一任务、统一结果和适配化模型接入，将系统演进为可持续扩展的综合学术 AI 鉴伪平台。

4 模块设计

4.1 模块划分原则

本系统的模块划分不按代码目录机械展开，而是按业务职责、数据边界和协作关系进行组织。其核心目标是将当前代码中已经存在的用户、组织、资源、检测、审核、通知和日志能力沉淀为稳定模块，同时为论文检测、评审文本检测和统一报告等迭代需求预留扩展空间。

在划分过程中遵循以下原则：

- **围绕业务能力划分模块**。模块对应稳定业务职责，而不是简单对应前端页面或后端文件夹；
- **高内聚、低耦合**。单个模块内部尽量完成同一类业务闭环，不直接依赖过多外部模块细节；
- **统一入口、差异化实现**。资源管理、检测任务管理和结果输出尽量统一，不同检测对象的差异通过专用检测模块或适配器处理；
- **兼顾现有实现与后续演进**。模块设计既要能映射当前 Django 后端、Vue 前端、Celery 任务和 AI 服务实现，也要能支撑下一阶段新增论文与评审能力。

4.2 模块总体关系类图

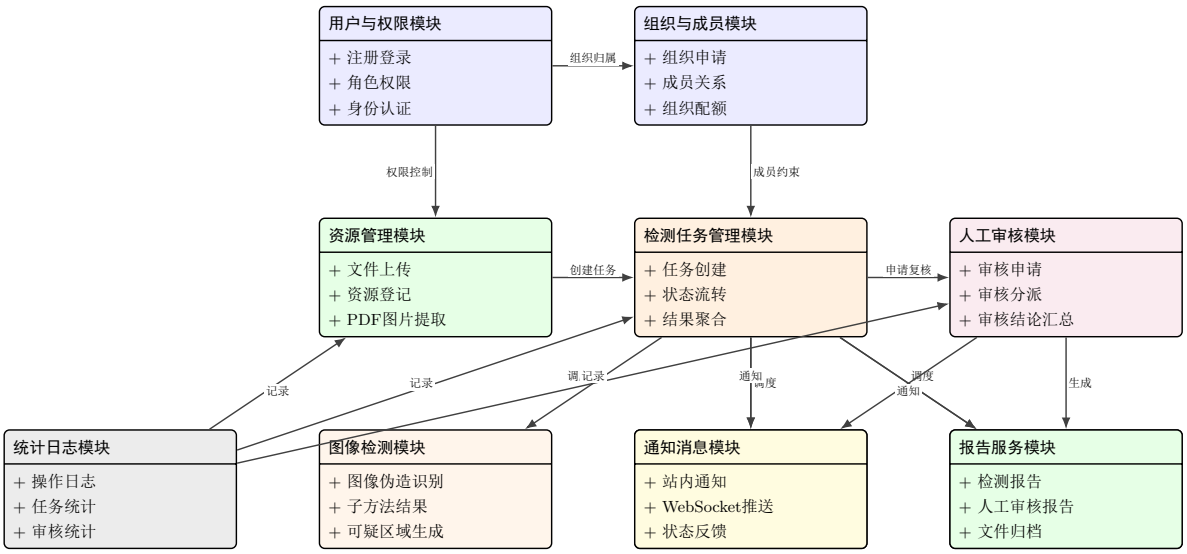

图 3: 系统模块关系 UML 类图

图 4-1 展示了本系统在模块层级上的主要组成和依赖关系。用户与权限模块、组织与成员模块和资源管理模块构成平台基础能力；检测任务管理模块作为任务中枢，统一调度图像检测、论文检测和评审检测三个专业检测模块；自动检测结果可进一步进入人工审核模块进行复核；报告服务模块对检测和审核结果进行汇总输出；通知消息模块和统计日志模块则贯穿全流程，承担状态反馈与治理支撑职责。

4.3 模块总览

模块名称	主要职责	主要依赖
用户与权限模块	负责注册登录、身份认证、角色区分、权限校验和账户状态管理	组织与成员模块、通知消息模块
组织与成员模块	负责组织申请、组织信息维护、邀请码管理、成员关系和组织级配额管理	用户与权限模块、统计日志模块
资源管理模块	负责文件上传、资源登记、分类标记、图片提取和资源查询	用户与权限模块、检测任务管理模块、文件存储

检测任务管理模块	负责任务创建、资源绑定、状态流转、异步调度、结果聚合和任务级报告关联	资源管理模块、各检测模块、报告服务模块、通知消息模块
图像检测模块	负责图像预处理、图像真伪检测、子方法结果汇总和可疑区域生成	检测任务管理模块、AI 推理服务
论文检测模块	负责论文解析、文本切分、AIGC 风险分析和论文结果封装	检测任务管理模块、AI 推理服务
评审检测模块	负责评审文本分析、模板化倾向识别和生成式痕迹分析	检测任务管理模块、AI 推理服务
人工审核模块	负责审核申请、审核分派、人工评分、审核结论汇总和人工审核报告生成	检测任务管理模块、用户与权限模块、通知消息模块
报告服务模块	负责任务报告和人工审核报告生成、下载与归档	检测任务管理模块、人工审核模块、文件存储
通知消息模块	负责站内通知、实时推送、状态更新提醒和审核结果通知	用户与权限模块、检测任务管理模块、人工审核模块
统计日志模块	负责操作日志、任务统计、审核统计和后台治理支撑	全部业务模块

4.4 用户与权限模块

用户与权限模块是整个平台的访问控制入口，负责用户注册、登录、身份认证、角色管理和基础权限校验。结合当前后端实现，该模块主要围绕 `User` 模型展开，并通过角色字段与权限位控制发布者、审核者和管理员在上传、提交、发布和审核等操作上的差异。

该模块的主要输入包括注册信息、登录凭证、邀请码和用户会话请求，主要输出包括认证结果、用户身份信息、角色信息和权限判断结果。由于用户权限与组织归属密切相关，因此该模块与组织与成员模块存在紧密协作关系。

从迭代角度看，该模块后续还需要继续承担统一权限入口职责，即论文检测、评审检测、模型配置和综合治理等新功能都不应绕过该模块单独实现权限控制。

4.5 组织与成员模块

组织与成员模块负责组织生命周期和组织内部协作关系管理，包括组织申请、组织审核、组织信息维护、管理员归属、邀请码分发、发布者与审核者关系维护，以及组织级配额管理等功能。当前代码中的 `OrganizationApplication`、`Organization`、`InvitationCode` 和 `PublisherReviewerRelationship` 等实体已经为该模块提供了实现基础。

该模块的输入主要包括组织申请资料、组织基本信息、成员关系调整请求和邀请码使用请求，输出则包括组织状态、成员归属关系和组织级资源使用能力。由于系统的用户角色和审核协作链条都依赖组织结构，因此该模块是平台治理能力的基础。

后续若要支持更复杂的机构治理，例如多级管理员、模型使用额度配置和组织维度统计看板，也应继续在该模块上扩展，而不是散落到各业务模块中实现。

4.6 资源管理模块

资源管理模块负责系统中所有待检测内容的接入、登记和管理，是检测业务链路的起点。当前实现已经围绕文件管理、图片上传和 PDF 中图片提取建立了较完整的图像资源处理链路，后续则需要把这一能力统一提升为面向图像、论文和评审文本三类资源的资源接入能力。

在当前代码中，该模块可映射到 `FileManagement`、`ImageUpload` 及相关上传接口和资源查询接口。其核心职责包括：

- 接收用户上传的原始文件或文本；
- 记录资源元数据、分类标签、来源信息和所属组织；
- 对 PDF 类资源执行图片提取等预处理；
- 为后续检测任务提供统一的资源引用和状态基础。

资源管理模块对外输出的是“可被检测任务引用的资源对象”，而不是直接输出检测结果。这一点有助于保持上传流程与检测流程的分离，使后续支持资源复检、批量检测和多类型资源扩展更容易实现。

4.7 检测任务管理模块

检测任务管理模块是整个系统的业务中枢，负责将资源接入、检测执行、状态流转、结果聚合和报告输出串联为一条完整链路。当前代码中的 `DetectionTask`、`DetectionResult`、`SubDetectionResult`，以及 Celery 任务和任务状态通知逻辑，共同构成了该模块的实现基础。

该模块的主要职责包括：

- 创建检测任务并绑定待检测资源；
- 初始化任务状态、检测方式和报告占位信息；
- 根据任务类型将请求投递到异步任务系统；
- 汇总单资源结果和子方法结果；
- 在任务完成后触发报告生成与通知发送。

在后续迭代中，该模块需要从“图像任务中心”提升为“统一检测任务中心”。也就是说，无论是图像检测、论文检测还是评审检测，都应尽量通过该模块进入统一状态机和统一任务生命周期，而不在外围额外构建平行任务体系。

4.8 图像检测模块

图像检测模块是当前系统中实现最成熟的专业检测模块，负责学术图像伪造识别、多检测方法执行、可疑区域生成和图像级结果输出。当前后端通过任务模块、AI 服务调用逻辑和 `SubDetectionResult` 子结果结构，已经具备图像多方法综合检测基础。

该模块的主要输入是图像资源、图像任务上下文和检测配置，输出则包括图像级真假判断、置信度、子方法概率、掩码图、可视化产物和图像级说明信息。由于图像检测往往耗时较长，因此它不直接面向前端同步返回，而是通过检测任务管理模块与异步任务系统协同完成执行。

从设计上看，该模块是后续其他检测模块的参考模板。论文检测和评审检测虽然算法不同，但在“接收任务上下文、执行检测、回写结构化结果”这一模式上可以沿用相同接口思想。

4.9 论文检测模块

论文检测模块是本轮迭代新增的重点模块之一，负责对整篇论文进行解析、切分和 AIGC 风险分析，并输出论文级别和段落级别的检测结果。当前代码中尚未形成完整实现，因此本章对该模块主要给出目标结构设计。

该模块的主要职责包括：

- 解析论文文件内容并提取结构化文本；
- 按章节、段落或语义单元进行文本切分；
- 调用对应模型评估生成式痕迹或文本异常模式；
- 将段落级检测结果汇总为论文级风险结论；
- 为报告服务模块提供可直接引用的结构化结果。

该模块不应复用图像检测模块中的结果结构细节，而应在统一任务框架下采用专属结果载荷。这样既能共享任务生命周期，又能避免论文结果被迫适配为图像式结构。

4.10 评审检测模块

评审检测模块面向 Review 文本，重点关注模板化倾向、重复性表达、生成式语言痕迹和不自然的学术评审模式。该模块与论文检测模块同属文本类检测能力，但其输入对象、特征关注点和输出形式不同，因此应作为独立模块设计，而不是简单并入论文检测模块。

该模块的主要输入是评审文本和任务上下文，输出包括模板化风险、生成式风险、可疑片段说明和评审级结论。其在系统中的位置与图像检测模块、论文检测模块并列，统一由检测任务管理模块调度。

这样设计的好处是：未来如果评审检测引入更细粒度规则引擎、相似评审比对或多轮评审一致性分析，不需要对论文检测模块进行破坏性改造。

4.11 人工审核模块

人工审核模块负责对自动检测结果进行人工复核，是保证平台可信度和可解释性的关键模块。当前代码已经围绕 `ReviewRequest`、`ManualReview`、`ImageReview` 和审核相关视图接口实现了较完整的图像人工审核链路。

该模块的主要职责包括：

- 接收用户发起的人工审核申请；
- 由管理员审批并分派审核人员；
- 支持审核人员逐项查看自动检测结果并提交人工判断；
- 汇总多名审核人员意见形成审核结论；
- 在审核完成后生成人工审核报告并反馈给申请方。

从演进角度看，该模块后续不应只服务图像资源，还应逐步扩展为“统一人工复核模块”。也就是说，论文和评审文本在必要时也应能纳入人工审核流程，只是其评分维度和审核界面可能与图像不同。

4.12 报告服务模块

报告服务模块负责将检测结果和人工审核结果整理为对用户可见、可下载、可归档的正式输出。当前后端已经通过 `report_generator.py` 具备任务检测报告和人工审核报告的 PDF 生成能力，因此该模块已有较明确的实现基础。

该模块的主要输入包括任务级结果、子检测结果、人工审核结论和基础元数据，输出包括任务报告、单资源报告、人工审核报告以及后续可能扩展的综合报告。由于报告本质上是对其他模块输出的汇总，因此该模块不负责产生原始检测结论，而负责对结论进行结构化表达、格式化展示和文件归档。

在后续迭代中，该模块需要从“图像任务报告生成器”演进为“统一报告服务”，使图像、论文、评审和人工审核结果都能通过统一模板策略输出。

4.13 通知消息模块

通知消息模块负责向用户和管理员反馈系统中的关键状态变化，包括任务开始、任务完成、报告生成、审核申请、审核结论和系统公告等。当前实现已经通过 `Notification` 模型、通知接口和 Django Channels WebSocket 消费者支持实时消息推送。

该模块的价值不在于保存单条消息本身，而在于作为系统状态外显层，让异步任务和长时审核流程能够及时反馈给前端用户。对于以异步检测为核心的平台而言，通知模块是体验闭环的一部分，而不是可有可无的附属功能。

4.14 统计日志模块

统计日志模块负责记录平台运行过程中的关键操作和业务指标，为管理端统计分析、问题追踪和后续审计提供支撑。当前代码中的 `Log` 模型、管理端统计接口和部分看板接口已构成该模块的基础。

该模块的主要职责包括：

- 记录上传、检测、审核申请、人工审核等关键操作日志；
- 支持按用户、组织、任务和时间范围进行查询；
- 为管理端提供任务统计、审核统计和趋势分析数据；
- 为后续论文日志、评审日志和模型使用日志扩展提供统一治理入口。

考虑到本项目后续治理能力会不断增强，日志与统计模块不应被视为简单的后台附属页面，而应作为平台可追溯性和可管理性的基础模块持续完善。

4.15 模块协作关系说明

从整体协作看，系统模块之间形成了较清晰的主干链路。用户与权限模块和组织与成员模块提供访问与归属基础；资源管理模块负责将检测对象接入平台；检测任务管理模块负责统一编排自动检测流程；图像检测、论文检测和评审检测模块分别提供专业检测能力；人工审核模块在需要时对自动结果进行补充复核；报告服务模块负责输出正式文档；通知消息模块和统计日志模块则在全过程中提供反馈和治理支撑。

这一协作结构的优势在于：

- 主业务链条清晰，便于后续逐步实现论文检测和评审检测；
- 模块间职责边界相对稳定，降低了“功能增长即结构失控”的风险；
- 与当前代码已有实现具有较强映射关系，便于后续详细设计与实际开发衔接。

4.16 本章小结

本章从业务职责角度对系统进行了模块化拆分，并结合当前代码结构与后续迭代目标，给出了模块边界、依赖关系和演进方向。后续详细设计可在本章基础上进一步细化每个模块的实体结构、接口定义、时序流程和异常处理策略。

5 数据设计

5.1 数据设计目标

数据设计需要同时满足统一资源抽象、现有图像业务兼容、异步任务追踪和后续扩展支持等要求。

5.2 核心实体

实体名称	说明
User	系统用户，包含普通用户、审核者、组织管理员和超级管理员。
Organization	用户所属组织及其资源配额、管理关系和邀请关系。
Resource	统一学术资源抽象，用于表示图像、论文和Review。
DetectionTask	检测任务主体，用于关联用户、资源集合、状态与任务参数。
DetectionResult	检测结果主体，用于承载自动检测输出及其状态。
ManualReview	人工审核任务与审核执行记录。
Report	检测或审核报告文件及其元数据。
Log	操作日志、检测日志、审核日志及管理日志。
Notification	系统消息与状态通知。

5.3 实体关系说明

本节后续将补充核心实体关系图，重点体现用户、组织、资源、检测任务、检测结果、人工审核、报告和日志之间的映射关系。

5.4 状态设计

- 资源状态：已上传、已解析、可检测、已归档等；
- 任务状态：待处理、处理中、已完成、失败、待人工审核等；
- 审核状态：待审批、审核中、已完成、已拒绝等；
- 通知状态：未读、已读。

5.5 兼容与演进说明

考虑到当前代码实现中已有围绕图像任务构建的数据模型，后续详细设计阶段需要明确哪些现有表结构继续复用，哪些通过扩展字段过渡，哪些需要新增统一资源抽象表。

6 接口设计

6.1 接口设计原则

- 对前端提供统一风格的REST接口；
- 对长时任务提供可轮询或可推送的状态查询能力；
- 对不同资源类型采用统一任务接口与差异化结果接口结合的方式；
- 对内部AI服务调用进行适配封装，避免前端或业务层直接依赖具体模型实现。

6.2 外部接口分类

- 用户认证接口；
- 资源上传与查询接口；
- 检测任务提交与状态接口；
- 检测结果查询与报告下载接口；
- 人工审核相关接口；
- 通知与消息接口；
- 管理端治理与统计接口。

6.3 内部接口分类

- 后端业务服务与异步任务系统之间的调度接口；
- 后端与AI推理服务之间的检测调用接口；
- 后端与文件存储之间的资源访问接口；
- 后端与消息通道之间的状态推送接口。

6.4 接口分层说明

本节后续将根据详细设计需要补充关键接口的职责、输入输出和调用关系，但不在概要设计阶段展开到字段级参数说明。

7 关键流程设计

7.1 资源上传与检测流程

本流程描述用户上传资源、后端创建任务、异步系统调度检测、AI服务返回结果以及系统回写状态和结果的全过程。

7.2 图像检测流程

本流程重点描述图像提取、图像批处理、图像检测、可疑区域标注、结果聚合和图像报告生成。

7.3 论文检测流程

本流程重点描述论文解析、文本切分、段落级分析、事实性说明生成和论文检测结果整合。

7.4 Review检测流程

本流程重点描述Review文本输入、模板化倾向分析、AI生成倾向分析与结果展示。

7.5 人工审核流程

本流程重点描述人工审核申请、管理员审批、审核任务下发、审核执行、审核结果汇总和人工审核报告生成。

7.6 通知与状态同步流程

本流程重点描述检测状态、审核状态和系统公告在前端与后端之间的同步机制。

7.7 流程图说明

本节后续补充活动图或时序图，建议至少覆盖上传检测、人工审核和报告生成三条主流程。

8 运行与部署设计

8.1 部署结构

系统部署上至少包含以下节点：

- Web前端部署节点；
- Django后端服务节点；
- Redis与异步任务处理节点；
- AI推理服务节点；
- MySQL数据库节点；
- 文件存储目录或对象存储节点。

8.2 运行机制

用户请求首先到达前端和后端，短请求由后端同步处理，长时检测任务通过异步队列提交到任务系统，再由AI推理服务执行，最终检测结果、日志和报告回写数据库与文件存储，并通过轮询或消息机制反馈到前端。

8.3 部署约束

- AI推理服务对运行环境、模型文件和GPU资源有依赖；
- 异步任务系统依赖消息中间件与稳定的任务状态存储；
- 文件上传、结果图片和PDF报告需要统一的存储路径管理；
- 生产部署时需明确静态资源、媒体文件、WebSocket与API代理配置。

9 质量属性设计

9.1 性能设计

系统需要支持批量上传、异步检测、状态同步和报告生成等场景，避免长时任务阻塞常规交互请求。

9.2 安全设计

系统需采用身份认证、角色权限控制、敏感数据保护、文件上传校验和关键操作审计等机制保障运行安全。

9.3 可维护性设计

系统应通过模块化拆分、统一抽象、接口适配和日志留痕提升维护效率，并降低新模型接入或新资源类型扩展带来的改动范围。

9.4 可扩展性设计

系统在设计上需要支持：

- 新增检测对象；
- 新增检测模型；
- 扩展报告内容；
- 扩展管理统计维度；
- 扩展人工审核规则。

9.5 可靠性设计

系统需具备异常提示、任务重试、日志追踪和必要的数据恢复能力，以降低网络异常、模型失败或服务中断对业务流程的影响。

10 设计约束与待定问题

10.1 当前约束

- 现有实现以学术图像检测为核心，论文与Review能力仍需扩展；
- 当前数据模型与接口设计中存在图像中心化特征；
- AI推理链路与后端任务链路的耦合仍需在后续设计中进一步梳理；
- 部分管理能力和模型配置能力尚未形成统一抽象。

10.2 待定问题

- 统一资源抽象与现有图像表结构的兼容迁移方案；
- 论文检测与Review检测的结果数据结构；
- 综合报告中多类型资源结果的整合方式；
- 模型管理配置在管理端中的可视化表达方式；
- 论文与Review专用日志的存储粒度和查询维度。

10.3 后续工作

后续将基于本概要设计继续细化详细设计，包括数据库设计、接口字段设计、模块内部类与函数设计、关键流程图、异常处理策略和测试设计。